AirGated

Technical Report

Offline Classroom Attendance with Device Binding and KYC-Style Face Liveness

Version	2.0
Date	30 September 2026
Author	Aakshant Kumar
Status	Working prototype — first multi-device live trial completed
Supersedes	Progress Report & Critical Analysis (1–4 August 2026)
Code baseline	git master @ 50d677e — server.py, face_verify.py, certs.py, 31 automated tests

Contents

Contents 2
1. Executive Summary 3
2. Version History and Scope 3
3. System Architecture 4
4. Captive-Portal Detection 4
5. Protocol 5
5.1 Enrollment 5
5.2 Check-in 6
5.3 Challenge and signed message 6
6. Verification Pipeline 7
7. KYC-Style Face and Liveness Verification 8
7.1 Capture 8
7.2 Models 8
7.3 Blink counting 8
7.4 Identity match and consistency 8
7.5 Server authority 9
8. Latency Gate: Analysis and Recalibration 9
8.1 Trial evidence 9
8.2 New estimator 9
9. Enrollment Lifecycle and Recovery 10
10. Data Model, Audit Log and Dashboard 10
11. Live Trial — 30 September 2026 10
11.1 Findings from the trial 11
12. Verification and Testing 12
12.1 Mutation testing 12
13. Status Against the August Findings 12
14. Open Issues and Recommendations 13
14.1 Next trial plan 14
Appendix A. HTTP and WebSocket Interface 15
Appendix B. Configuration Constants 15
Appendix C. Repository Layout 15

1. Executive Summary

AirGated turns an instructor's laptop into a self-contained, offline attendance authority. Students join a local Wi-Fi network, are steered to a captive portal, and prove — in a single check-in — that they are on the classroom network, holding the phone they enrolled, and physically present as the enrolled person. This report documents version 2.0 of the system: the architecture, the check-in protocol, the design of the new KYC-style face and blink verification, the evidence from the first live multi-device trial, and the status of every finding raised in the August progress report.

What changed since the progress report. Every code-level finding from the August report has been addressed. The key-extractability regression (4.1) is closed; the replay gap (Section 5 of that report) is closed by server-issued single-use challenges; the missing captive-portal popup (4.2b) was traced to a concrete root cause — the port-80 listener answered the OS connectivity probe with 204, telling the phone the network already had internet — and fixed; certificates now persist across restarts; dead prototypes are archived; an audit log and a read-only attendance dashboard exist; and the repository has 31 automated tests whose effectiveness was verified by mutation testing. Two previously unreported weaknesses were found and fixed during the work: the latency gate trusted an RTT value reported by the client, and nothing prevented a student from handing their enrolled phone to a friend. The second is closed by the new face-match and blink-liveness gate.

Live trial (30 September 2026). Three students enrolled three different phones and produced 17 recorded events: 3 enrollments and 4 check-ins accepted; 10 attempts rejected. The security controls behaved as designed — a check-in for one student from another student's phone was stopped by the MAC binding gate, and a face that did not match the enrolled student (similarity 0.204 against genuine scores of 0.715–0.879) was rejected. The trial also exposed two sources of genuine-user friction: the latency gate rejected two honest phones at about one metre (now fixed in code by changing the RTT estimator), and blink liveness rejected six attempts in which the student blinked but the server counted fewer blinks than requested. Liveness calibration is the most important open item.

> **Bottom line**
>
> The system is now functionally complete for a single-room trial and its security properties are backed by tests. Before a full-class deployment it needs (1) liveness calibration to cut false rejections, (2) an explicit room-presence signal, because Wi-Fi reach is not a room boundary, and (3) an access point that can hold a whole class — the Windows Mobile Hotspot is capped at eight clients.

2. Version History and Scope

Table 1. Document history.

Version	Date	Document	Summary
1.0	28 May 2026	AirGated_Report.md	Project write-up: GPS phase abandoned, device-binding cryptography, network hardening.
1.x	1–4 Aug 2026	Progress Report & Critical Analysis	Ten critical findings; route-ordering bug fixed; replay gap and key regression identified.
2.0	30 Sep 2026	This report	All code findings addressed; KYC face + blink liveness; live three-phone trial; test suite.

In scope: the attendance pipeline (captive portal, enrollment, check-in, verification gates, audit and dashboard), its security model and its test evidence. **Out of scope:** the Groq-backed exam generator, which shares the server process but is unrelated to attendance, except where it affects security (Section 13).

3. System Architecture

All components run on the instructor laptop; no attendance decision depends on the internet. The laptop exposes three listeners on the hotspot interface and uses three local resources.

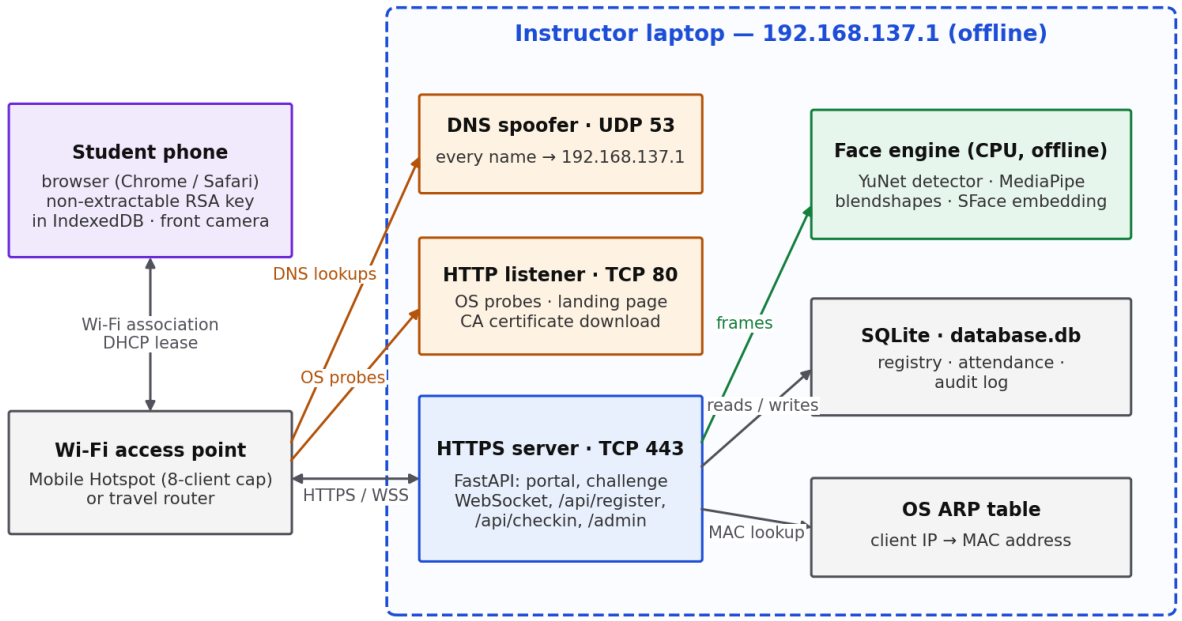


Figure 1. Deployment topology. Every DNS name resolves to the laptop, so all phone traffic is handled locally; the face engine, database and ARP table are only reachable by the HTTPS server.

Table 2. Runtime components.

Component	Port / file	Responsibility
DNS spoofer	UDP 53	Answers every query with 192.168.137.1 so connectivity probes and browsing reach the laptop.
HTTP listener	TCP 80	Answers OS connectivity probes (redirect or success, per client state); serves the landing page shown in the OS sign-in popup and the classroom CA certificate.
HTTPS server (FastAPI)	TCP 443	Student portal, challenge WebSocket, /api/register, /api/checkin, localhost-only /admin dashboard. Catch-all redirect registered last.
Face engine	face_verify.py	Offline face detection (YuNet), blink scoring (MediaPipe Face Landmarker blendshapes) and identity embedding (SFace).
Certificate manager	certs.py	Creates an ECDSA P-256 classroom CA (10 years) and a server certificate (825 days) once; reuses them until 30 days before expiry or an IP change.
Persistence	database.db	Device registry, attendance records, audit log (SQLite). Not committed to version control.

A secure origin (HTTPS) is not optional: browsers only expose `crypto.subtle` (device keys) and `getUserMedia` (camera) to secure contexts, which is why the portal is served on 443 and the plain-HTTP listener only handles detection and hand-off.

4. Captive-Portal Detection

When a phone joins Wi-Fi, the operating system fetches a well-known plain-HTTP URL and decides from the answer whether the network has internet, needs a sign-in, or is broken. In the August build the port-80 listener returned `204 No Content` to every probe. A 204 is the "internet works" answer, so phones concluded no sign-in was needed and never raised the popup — the root cause of Finding 4.2b. Subsequent

HTTPS checks then failed against the self-signed certificate, which is why the phone also reported "No internet connection".

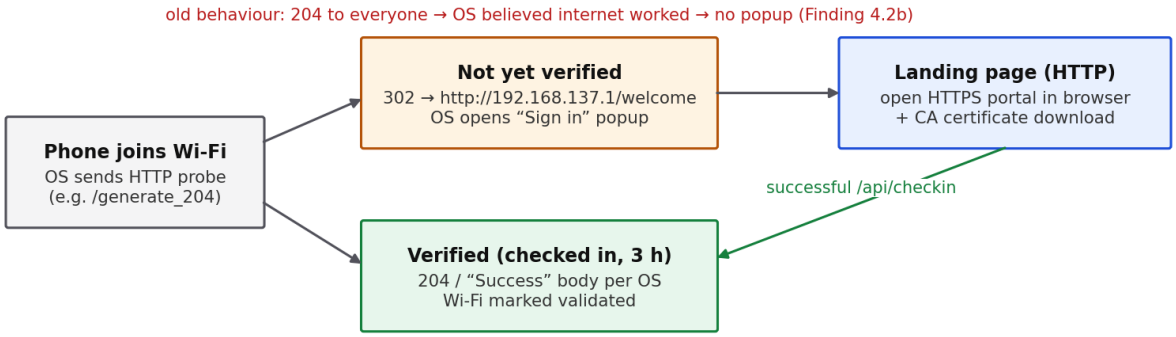


Figure 2. Probe handling in v2.0. The answer now depends on whether the client IP has completed a check-in in the last three hours.

Table 3. Connectivity probes recognised by the server.

Platform	Probe path	Before check-in	After check-in
Android / ChromeOS	/generate_204, /gen_204	302 → /welcome	204
iOS / macOS	/hotspot-detect.html, /library/test/success.html	302 → /welcome	200 "Success" page
Windows	/connecttest.txt, /ncsi.txt	302 → /welcome	200 "Microsoft Connect Test" / "Microsoft NCSI"
Firefox	/success.txt, /canonical.html	302 → /welcome	200 expected body

The OS sign-in window is a restricted web view that generally cannot open the camera, so the landing page directs the student to open `https://192.168.137.1/generate-captive-window` in Chrome or Safari and offers the classroom CA for one-time installation.

5. Protocol

5.1 Enrollment

Enrollment binds three things to a roll number: a device key, a link-layer address and a face.

- The browser generates an RSA-2048 (RSASSA-PKCS1-v1_5, SHA-256) key pair with `extractable: false`. The private key is a `CryptoKey` object stored in IndexedDB; its bytes are never readable by page script, so it cannot be copied to another device (closes Finding 4.1).
- The browser opens the challenge WebSocket with `purpose=register`, receives a nonce and a random blink count, records a short clip while the student blinks, and signs the canonical message (Section 5.3) with the new private key — proof of possession.
- The server verifies the signature against the submitted public key, runs liveness and extracts the face embedding, reads the MAC from its ARP table, and stores public key, MAC and embedding.
- Only after the server accepts does the browser persist the private key, so a rejected enrollment never leaves an orphaned key that would make the page believe it is enrolled.

5.2 Check-in

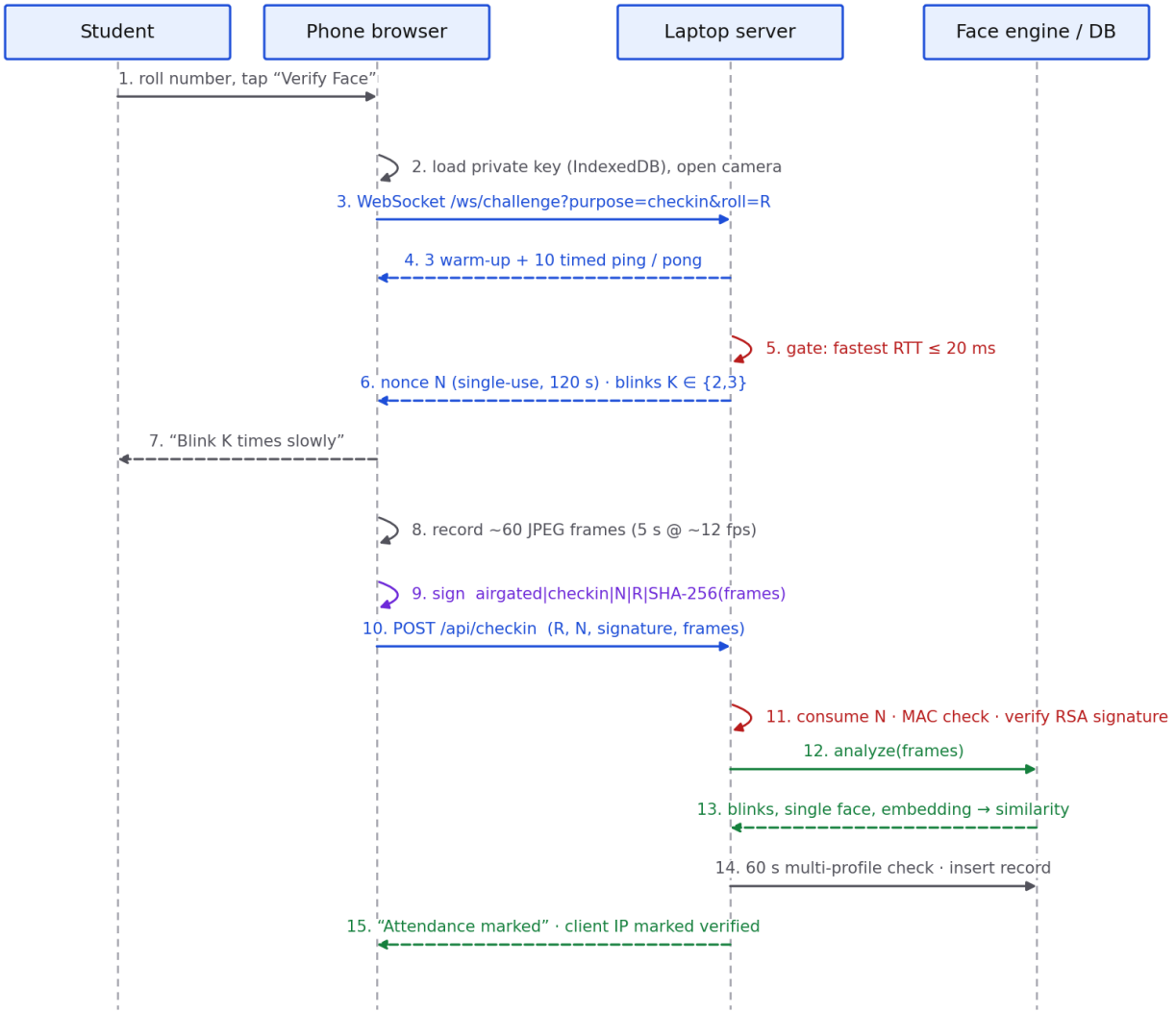


Figure 3. Check-in sequence. Dashed arrows are responses; the red self-calls are server-side gates.

5.3 Challenge and signed message

Each registration or check-in consumes exactly one server-issued challenge. Challenges are held in memory and are bound to the request that asked for them.

Table 4. Challenge properties.

Property	Value	Purpose
Nonce	128-bit random (secrets.token_hex(16))	Unpredictable, cannot be pre-computed
Binding	purpose, roll number, client IP	A nonce for one student, device or purpose is useless for another
Lifetime	120 s, deleted on first use	Replaying a captured request fails (“already used”)
Carried data	server-measured RTT, required blink count K ∈ {2, 3}	The client never reports latency; K is revealed only after the nonce

The device signs the UTF-8 string below. Because the SHA-256 digest of the exact JPEG frames is inside the signed message, neither the nonce nor the video can be swapped after signing.

```
airgated|<purpose>|<nonce>|<roll_number>|<sha256_hex(frame_1 ‖ frame_2 ‖ … ‖ frame_n)>
```

6. Verification Pipeline

A check-in is accepted only if it passes every gate in order. Each rejection returns a message to the student and writes a typed event to `airgated_audit_log`.

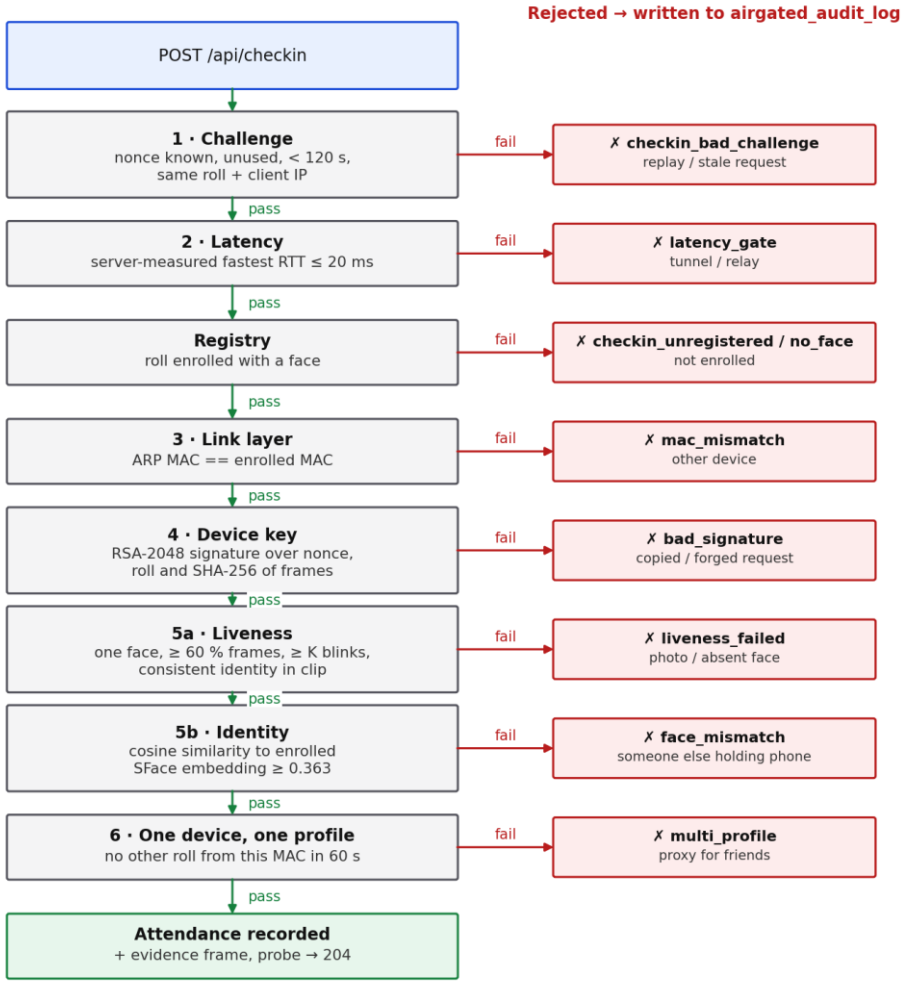


Figure 4. The verification pipeline as implemented in verify_checkin(). Every red box is a distinct audit event visible on the dashboard.

Table 5. What each gate proves and why the client cannot forge it.

Gate	Proves	Why the client cannot fake it
1 Challenge	The request is fresh and intended for this roll and this client	The server created the nonce and deletes it on use
2 Latency	Direct link to the classroom network (no relay or tunnel)	RTT is measured by the server over the WebSocket
3 Link layer	Request comes from the enrolled network interface	MAC is read from the laptop's own ARP table
4 Device key	Request comes from the enrolled phone's secret key	Key is non-extractable; signature covers the frames
5 Face	A live person matching the enrolled face holds the phone	Frames are analysed on the laptop, not on the phone
6 One device, one profile	One phone is not marking several students present	Checked against the server's own records

7. KYC-Style Face and Liveness Verification

The face gate addresses the threat that device binding cannot: a present student carrying an absent friend's enrolled phone. It follows the pattern of remote identity verification (KYC) flows — an active liveness challenge plus a comparison against an enrolled reference — but runs entirely offline on the laptop CPU.

7.1 Capture

After the challenge is issued the portal shows the live front-camera view, a three-second countdown and the instruction “Blink K times slowly”, then records for 5 s. Frames are scaled to 320 px wide and JPEG-encoded at quality 0.8 every 80 ms (≈12.5 fps, ≈60 frames, typically 5–12 KB each). The server accepts at most 120 frames of at most 300 KB each.

7.2 Models

Table 6. Face models (downloaded once by fetch_models.py; SHA-256 pinned).

Model	File	Size	Role
YuNet (OpenCV Zoo, 2023)	face_detection_yunet_2023mar.onnx	0.23 MB	Counts faces per frame; supplies landmarks for alignment
MediaPipe Face Landmarker (float16)	face_landmarker.task	3.6 MB	eyeBlinkLeft / eyeBlinkRight blendshape scores
SFace (OpenCV Zoo, 2021)	face_recognition_sface_2021dec.onnx	36.9 MB	128-dimensional identity embedding

7.3 Blink counting

For each frame containing exactly one face, the blink score is the mean of the two eye-blink blendshapes (0 = open, 1 = shut). A blink is counted on an open → closed → open transition using hysteresis: “closed” requires ≥ 0.45 and “open” requires ≤ 0.25. The gap prevents one slow blink from being counted twice, squints never cross the closed threshold, and a clip must show open eyes first, so a clip that starts mid-blink does not gain a free count.

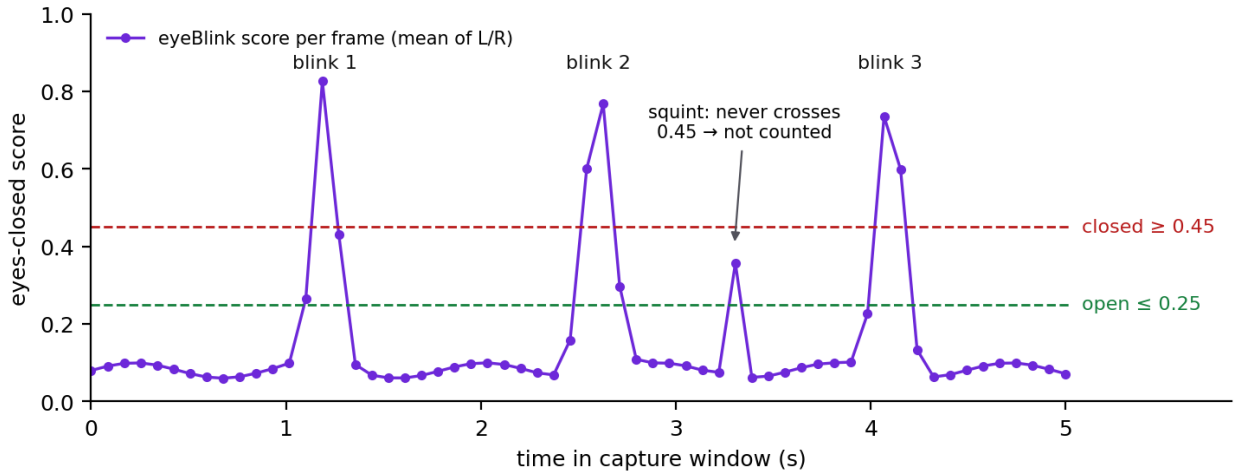


Figure 5. Illustrative blink-score trace over a 5 s capture: three blinks counted, one squint ignored. At ≈12 fps a blink's closed phase spans only one to three frames.

7.4 Identity match and consistency

SFace embeddings are taken from up to 12 open-eye frames and L2-normalised. Liveness additionally requires at least three such frames, at least 60 % of frames containing a face, and no frame with more than one face. Each sampled embedding must have cosine similarity ≥ 0.363 to the clip's mean embedding, so a clip that switches people part-way through fails. The mean embedding is then compared with the enrolled

one; 0.363 is OpenCV's published operating point for SFace and has not yet been re-calibrated locally (Section 12).

7.5 Server authority

The phone only records. Every liveness and identity decision is made on the laptop from the signed frames, so editing the page's JavaScript cannot make a check-in pass. Residual risk: an attacker who holds the enrolled phone and a video of the student blinking the requested number of times could replay it into the camera (Section 13).

8. Latency Gate: Analysis and Recalibration

The latency gate distinguishes a direct link to the laptop from a relayed or tunnelled one. In v2.0 the RTT is measured by the server itself over the challenge WebSocket; the August design accepted a `current_rtt` value submitted by the client, which any user could set to 1 ms, and timed only the last of its three pings.

8.1 Trial evidence

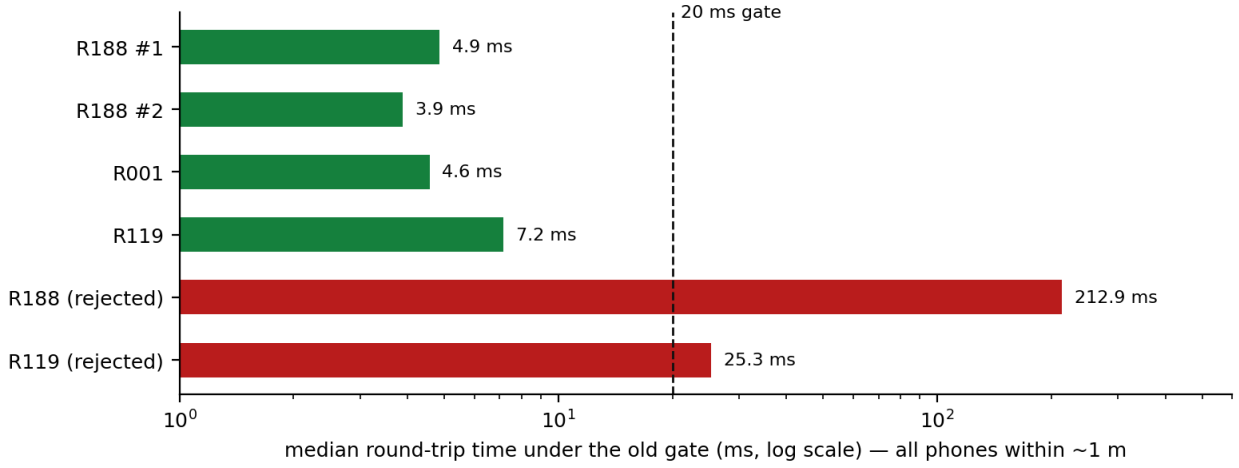


Figure 6. Server-measured RTT during the 30 September trial (old estimator: median of five pings). All phones were about one metre from the laptop.

Accepted check-ins measured 3.9–7.2 ms, yet two honest attempts were rejected at 212.9 ms and 25.3 ms. Radio propagation over one metre takes a few nanoseconds, so distance cannot explain the difference. The cause is Wi-Fi power saving: an idle phone radio sleeps between beacons (typically ~100 ms apart), so any ping that arrives while it sleeps waits for the next wake-up. With only five samples, two or three such waits push the median over the threshold.

8.2 New estimator

- Three warm-up pings are sent and discarded; they wake the radio out of power save.
- Ten timed pings follow; the gate uses the **minimum** RTT and the median is recorded for diagnostics.
- Sleep, contention and retransmission can only make a ping slower. A relay or tunnel cannot make its fastest ping quicker than its real path, so the minimum is the robust statistic for “is this a direct local link”.
- Threshold unchanged at 20 ms. Unit test `test_rtt_gate_ignores_power_save_spikes` uses spike patterns from the trial.

> **Important limitation**
>
> No RTT threshold can measure distance inside Wi-Fi range. At these scales the gate proves network membership without a relay, not presence inside the room: a phone in the corridor with a good signal measures the same as a phone on the front desk. Room presence needs a different signal (Section 14).

9. Enrollment Lifecycle and Recovery

The August build silently overwrote whatever roll was registered to a MAC whenever that MAC re-registered, downgrading the model to “trust the latest device on this MAC”. v2.0 replaces this with explicit rules:

Table 7. Re-enrollment rules.

Situation	Outcome	Audit event
Device's MAC is already enrolled to a different roll	Refused — one phone, one student	register_device_taken
Roll not enrolled	New enrollment	—
Roll enrolled with a face; new face matches (≥ 0.363)	Re-key: new phone and MAC accepted	reenroll_device
Roll enrolled with a face; face does not match	Refused	reenroll_face_mismatch
Roll enrolled before faces existed, same MAC	Face added to the existing enrollment	legacy_face_added
Roll enrolled before faces existed, different MAC	Refused — instructor reset required	legacy_upgrade_refused
Lost or replaced phone (instructor action)	Enrollment deleted from the dashboard; student enrolls again	instructor_reset

10. Data Model, Audit Log and Dashboard

Table 8. Database tables (created or migrated automatically at start-up).

Table	Key columns	Notes
airgated_device_registry	roll_number (PK), name, public_key_hex, hardware_mac, face_embedding (128 × float32), enrolled_at	One row per enrolled student
airgated_attendance_records	roll_number, timestamp, latency_ms, network_ip, network_mac, face_similarity, blinks_detected, evidence_file	One row per accepted check-in
airgated_audit_log	timestamp, event, roll_number, network_ip, network_mac, detail	Every rejection plus re-enrollment and reset events

Nonces and the set of verified client IPs are deliberately held in memory: a restart simply forces students to restart a check-in. The best open-eye frame of each accepted check-in is written to `evidence/` for review.

The instructor dashboard (`https://localhost/admin`, loopback only, refreshed every 30 s, selectable day) is read-only apart from enrollment reset. It shows: enrolled / present / absent counts and attendance rate; a bar chart of check-in failure reasons; a seven-day attendance trend; a “needs attention” list of students who failed and were never marked present; present and absent lists with face score, blink count and RTT; every failed check-in and enrollment with its reason; and the reset form for lost phones.

11. Live Trial — 30 September 2026

Setup. Instructor laptop running Windows Mobile Hotspot at 192.168.137.1; three students, each on their own phone, approximately one metre from the laptop; fresh database. Events 01:08–01:27 local time, taken directly from `airgated_attendance_records` and `airgated_audit_log`.

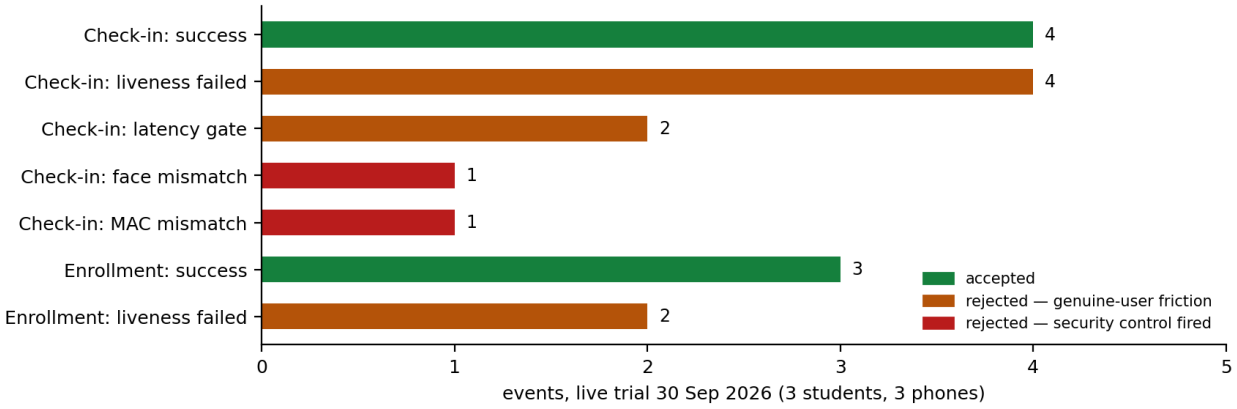


Figure 7. All recorded outcomes. Amber rejections are friction for genuine users; red rejections are security controls firing.

Table 9. Event log (roll numbers abbreviated to their last three digits).

Time	Roll	Event	Detail
01:08:49	188	☑ check-in	RTT 4.9 ms · face 0.720 · 2 blinks
01:09:04	188	liveness_failed	asked 2 blinks, saw 0
01:09:27	188	liveness_failed	asked 3 blinks, saw 2
01:09:51	188	☑ check-in	RTT 3.9 ms · face 0.715 · 2 blinks
01:14:57	001	register_liveness_failed	asked 3 blinks, saw 2
01:15:27	001	☑ check-in	RTT 4.6 ms · face 0.879 · 3 blinks
01:15:46	001	liveness_failed	asked 3 blinks, saw 1
01:16:02	001	liveness_failed	asked 2 blinks, saw 0
01:16:09	188	latency_gate	median RTT 212.9 ms (old estimator)
01:16:27	188	face_mismatch	similarity 0.204 on the enrolled phone
01:24:20	119	register_liveness_failed	asked 2 blinks, saw 1
01:24:37	119	latency_gate	median RTT 25.3 ms (old estimator)
01:24:55	119	☑ check-in	RTT 7.2 ms · face 0.823 · 5 blinks
01:26:50	119	mac_mismatch	request from roll 188's phone; enrolled MAC differs

11.1 Findings from the trial

- **Device binding works across real phones.** A check-in for roll 119 sent from the phone enrolled as roll 188 was rejected at the link-layer gate.
- **Face matching separates genuine from impostor with margin.** Genuine check-ins scored 0.715–0.879; the mismatched face on roll 188's phone scored 0.204. The 0.363 threshold sits between them, but three genuine and one impostor sample are far too few to calibrate it.
- **Latency false rejections (2 of 12 check-in attempts)** were caused by the median estimator and power-save spikes, not by distance. Fixed in commit 50d677e (Section 8); the new estimator still needs to be confirmed in the next trial.
- **Liveness false rejections are the dominant friction:** 4 of 12 check-in attempts and 2 of 5 enrollment attempts were rejected with fewer blinks counted than requested (0–2 counted versus 2–3 requested). Likely contributors are the ≈12 fps sampling (a blink's closed phase covers one to three frames, and a single blurred frame can stay below 0.45), students starting to blink during the countdown, and the rule that the eyes must reopen before the clip ends.
- **MAC randomisation was active on all three phones.** Each enrolled MAC has the locally-administered bit set (36:…, 1E:…, FA:…), i.e. all three were per-network random addresses. They stayed stable for the session, but some Android and iOS versions rotate them over time, which would break the MAC gate for that student until they re-enroll with their face.

- **Repeat check-ins are recorded.** Roll 188 checked in twice a minute apart; the dashboard counts distinct students, but records are not deduplicated per session.

12. Verification and Testing

The repository contains 31 automated tests (`python -m pytest tests`, about 20 s). Server tests drive the real FastAPI application in memory through HTTP and WebSocket calls, using a temporary SQLite database. Each simulated phone owns a real RSA-2048 key and signs messages in the browser's exact format, so signature verification is genuine. Two dependencies are substituted: the face engine (tests choose what it "sees": the enrolled face, a different face or too few blinks) and the ARP lookup (tests choose the MAC). The real face code and models are exercised separately in `tests/test_face_verify.py`.

Table 10. Test coverage by concern.

Concern	Tests	Examples
Challenge and replay	4	replayed signature rejected; nonce for another roll; expired nonce; server-measured RTT
Signatures	2	key from another device; signature over a different clip
Latency gate	2	gate rejects above threshold; power-save spikes do not cause rejection
Face and liveness decisions	2	face mismatch; too few blinks
Device binding and profiles	4	MAC mismatch; one MAC two rolls in 60 s; device cannot take a second roll; face-verified re-enrollment
Portal, routing, captive probes	4	no key export / localStorage; catch-all does not swallow APIs; probe 302 → 204 on HTTPS and HTTP
Dashboard and reset	4	end-to-end enroll and check-in; loopback-only; statistics correct; reset enables a new phone
Face engine unit tests	9	6 blink-counting cases; embedding round trip; liveness reasons; real models reject blank frames

12.1 Mutation testing

To show that the tests detect real defects rather than merely pass, five deliberate bugs were injected into a throwaway copy of the code and the suite was run against each. Every mutant was detected.

Table 11. Mutation results.

Injected defect	Detected by
Nonce not deleted after use (dict.get instead of dict.pop)	test_replayed_signature_rejected
Face-similarity check removed	test_face_mismatch_rejected
Blink counter increments by two	4 cases of test_count_blinks
Probes answered 204 for every client (the original popup bug)	test_probe_redirects_until_verified, test_http_listener_probe_behaviour
MAC comparison disabled	test_mac_mismatch_rejected

Limits. Tests establish that the decision logic is correct. They do not establish that the thresholds suit real cameras and lighting, that each OS raises the sign-in popup, or that latency stays within budget with a full class — those require field trials.

13. Status Against the August Findings

Table 12. Disposition of every finding in the Progress Report & Critical Analysis.

Finding	Issue	Status	Resolution
4.1	Private key extractable, stored in localStorage	Fixed	Non-extractable CryptoKey in IndexedDB, saved only after server acceptance; regression test.

Finding	Issue	Status	Resolution
4.2	Catch-all route swallowed /api/register and /api/checkin	Fixed	Catch-all registered last with no exempt list; regression test.
4.2b	OS sign-in popup never appeared	Fixed in code	Root cause: probe answered 204. Probes now redirect until check-in; landing page on port 80. Per-OS confirmation to be recorded.
4.3	MAC randomisation; silent MAC-based overwrite	Mitigated	Face-verified re-enrollment, one-device-one-roll rule, instructor reset, onboarding guidance. Randomisation itself cannot be detected.
4.4	Proximity = network membership	Open	Documented; estimator made robust. Needs a room-presence signal (Section 14).
4.5	Hotspot capped at 8 clients	Hardware	Documented; configurable server IP for a travel router.
4.6	Plaintext Groq API key	Fixed	Moved to git-ignored .env; key never present in history. Rotation recommended.
4.7	Certificates regenerated every start	Fixed	Persistent CA and server certificate; CA downloadable for one-time trust.
4.8	Three overlapping implementations	Fixed	Prototypes moved to archive/ with a README; single live path.
4.9	No automated tests	Fixed	31 tests plus mutation testing.
4.10	Git out of date; DB and __pycache__ tracked	Fixed	History rewritten to remove database.db and __pycache__; .gitignore added.
§5	Hard-coded challenge, replay possible	Fixed	Single-use, bound, expiring server nonce; signature covers frames.
§7	No reviewable fraud record	Fixed	Audit log table and attendance dashboard.
New	Client-reported RTT trusted	Fixed	Server-side measurement over the WebSocket.
New	Friend checks in with an absent student's phone	Fixed	Face match + blink liveness at enrollment and every check-in.
New	RTT median rejects honest phones	Fixed in code	Warm-up + minimum of 10; awaiting trial confirmation.

14. Open Issues and Recommendations

Hardware constraints (the hotspot's client cap) are listed in Section 13. The items below are software issues, in priority order.

Table 13. Open software issues.

#	Priority	Issue	Recommended action
1	High	Liveness false rejections (6 of 17 attempts in the trial)	Log per-frame blink scores to calibrate thresholds; raise capture to 15–20 fps at lower resolution; start recording after the countdown ends; show a live blink counter as UX feedback while keeping the server authoritative; offer an immediate retry.
2	High	No room-presence proof (Wi-Fi reaches corridors)	Add a short rotating code shown on the projector (e.g. changes every 15 s) that the student enters and that is included in the signed message; forwarding is limited to people already on the classroom Wi-Fi within seconds.
3	Medium	Video replay of a blinking face on the enrolled phone	Add a randomised active challenge (turn head left/right, look up) scored from head-pose landmarks.
4	Medium	Face threshold not locally calibrated	Collect genuine/impostor pairs from the class under room lighting and choose the threshold from the observed distributions.
5	Medium	No rate limiting on challenges or APIs	Per-IP and per-roll attempt limits with back-off; cap outstanding challenges.
6	Medium	Exam generator reachable by every student; unescaped HTML in generated page	Restrict /generate-exam and /view-exam to loopback; sanitise model output.
7	Low	Dashboard protected only by loopback check	Add an instructor password if the dashboard is ever exposed beyond the laptop.
8	Low	Class-scale performance unmeasured	Load test 60+ concurrent check-ins; measure face-analysis time and

#	Priority	Issue	Recommended action
			latency-gate stability.
9	Low	Biometric data retention and consent	Consent screen at enrollment; retention period and automatic deletion of evidence/ images.
10	Low	Camera unavailable inside the OS sign-in window; CA trust needed	Keep the browser hand-off; distribute the classroom CA during onboarding.
11	Low	Housekeeping	Deduplicate repeat check-ins per session; refresh AirGated_Report.md; convert test_qroq.py or remove it.

14.1 Next trial plan

- Re-run with the new RTT estimator and record best and median RTT for every attempt.
- Record per-OS popup behaviour (Android, iOS) and whether students needed to install the CA.
- Capture per-frame blink scores for every attempt to calibrate liveness (item 1).
- Move to a travel router and test with at least ten phones concurrently.

Appendix A. HTTP and WebSocket Interface

Method	Path	Listener	Description
GET	/welcome	HTTP 80	Landing page shown in the OS sign-in window
GET	/airgated-ca.crt	HTTP 80 / HTTPS 443	Classroom CA certificate download
GET	/generate-captive-window	HTTPS	Student portal (enrollment and check-in)
WS	/ws/challenge?purpose=&roll=	HTTPS	Server RTT measurement and challenge issuance
POST	/api/register	HTTPS	roll_number, name, public_key, nonce, signature_hex, frames[]
POST	/api/checkin	HTTPS	roll_number, nonce, signature_hex, frames[]
GET	/admin?date=YYYY-MM-DD	HTTPS, loopback	Attendance dashboard
POST	/admin/reset	HTTPS, loopback	Delete a student's enrollment (lost phone)
any	/{path}	HTTP and HTTPS	Connectivity probes per client state; everything else redirected to the portal

Appendix B. Configuration Constants

Constant	Value	Meaning
RTT_THRESHOLD_MS	20.0	Maximum fastest round-trip time
RTT_WARMUP_PINGS / RTT_PING_COUNT	3 / 10	Discarded warm-up pings / timed pings
CHALLENGE_TTL_SECONDS	120	Nonce lifetime
CAPTURE_MS	5000	Face clip length
BLINK_CHOICES	(2, 3)	Random blink count per challenge
BLINK_CLOSED_SCORE / BLINK_OPEN_SCORE	0.45 / 0.25	Blink hysteresis thresholds
MATCH_THRESHOLD	0.363	SFace cosine similarity for “same person”
MIN_FACE_FRAME_RATIO	0.6	Minimum share of frames containing a face
MAX_FRAMES / MAX_FRAME_BYTES	120 / 300 000	Upload limits
MULTI_PROFILE_WINDOW_SECONDS	60	One MAC, one roll window
VERIFIED_CLIENT_SECONDS	10 800	Probes answered as “online” after check-in
AIRGATED_LOCAL_IP (env)	192.168.137.1	Laptop address on the classroom network

Appendix C. Repository Layout

Path	Contents
server.py	DNS spoofer, HTTP listener, portal, challenge WebSocket, APIs, dashboard, exam generator
face_verify.py	Face detection, blink counting, embeddings, liveness decision
certs.py	Persistent classroom CA and server certificate
fetch_models.py	One-time, checksum-verified model download
templates/portal.html, templates/landing.html	Student portal (HTTPS) and sign-in window page (HTTP)
tests/	31 automated tests
archive/	Retired prototypes, not deployed
README.md	Setup, operation, diagrams and testing guide
Not versioned	.env, database.db, ssl_certs/, models/, evidence/, .venv/